

Nulos

Clase 06

IIC2413 - Sección 3

Prof. Cristian Riveros

`null` = *unknown or not applicable*

Satellites			
name	planet	discover	dyear
Luna	Tierra	<code>null</code>	<code>null</code>
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Ejemplo de inserción con `null`

```
INSERT INTO Satellites(name, planet, discover, dyear)
VALUES ('Luna', 'Tierra', NULL, NULL)
```

```
INSERT INTO Satellites(name, planet)
VALUES ('Luna', 'Tierra')
```

`null` = *unknown or not applicable*

Satellites			
name	planet	discover	dyear
Luna	Tierra	<code>null</code>	<code>null</code>
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Ejemplo de consulta con IS NULL

```
SELECT name, planet  
FROM Satellites  
WHERE discover IS NULL
```



name	planet
Luna	Tierra

null = *unknown or not applicable*

Satellites

name	planet	discover	dyear
Luna	Tierra	null	null
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Ejemplo de consulta con IS NOT NULL

```
SELECT name, planet  
FROM Satellites  
WHERE discover IS NOT NULL
```



name	planet
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter
Titán	Saturno
Triton	Neptuno

IS NULL o IS NOT NULL **verifica** si un valor es null o no.

¡No usen null!

*I couldn't resist the temptation to put in a **null reference**, simply because it was so easy to implement. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years.*

Tony Hoare
Ganador Turing Award

Ejemplo de como **no permitir** null en un atributo

```
CREATE TABLE Satellites ( name      CHAR(20), planet CHAR(20),  
                           discover CHAR(20) NOT NULL,  
                           dyear     INT NOT NULL )
```

¡Gracias! / ¿Preguntas?

¿por qué seguimos
entonces usando nulos?

	EmployeeID	Jobtitle	Age	FirstName	Salary
1	null	null	null	Holly	null
2	null	Salesman	null	null	43000
3	1001	null	30	Jim	null
4	1001	Salesman	null	null	45000
5	1002	null	29	Pam	null
6	1002	Receptionist	null	null	36000
7	1003	null	29	Dwight	null
8	1003	Salesman	null	null	63000
9	1004	null	31	Angela	null
10	1004	Accountant	null	null	47000
11	1005	null	32	Toby	null
12	1005	HR	null	null	50000
13	1006	null	35	Michael	null
14	1006	Regional Manager	null	null	65000
15	1007	null	32	Meredith	null
16	1007	Supplier Relations	null	null	41000
17	1008	null	38	Stanley	null
18	1008	Salesman	null	null	48000
19	1009	null	31	Kevin	null
20	1009	Accountant	null	null	42000

(ejemplo de tabla obtenida en la Web)

Outline

Nulos en SQL

Información incompleta

Outer Joins

Outline

Nulos en SQL

Información incompleta

Outer Joins

Problema: ¿qué significa null?

- Valor existe, pero no tengo la información.
- Valor no existe.
- Valor no se si existe o no.

¿cuál es el **semántica** de null?

Problema: ¿qué significa null?

Satellites

name	planet	discover	dyear
Luna	Tierra	null	null
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

¿son estas consultas equivalentes?

```
SELECT name, planet  
FROM Satellites
```



name	planet
Luna	Tierra
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter
Titán	Saturno
Triton	Neptuno

Problema: ¿qué significa null?

Satellites

name	planet	discover	dyear
Luna	Tierra	null	null
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

¿son estas consultas equivalentes?

```
SELECT name, planet  
FROM Satellites
```

```
SELECT name, planet  
FROM Satellites  
WHERE dyear = 1610 OR dyear <> 1610
```



name	planet
Luna	Tierra
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter
Titán	Saturno
Triton	Neptuno

¿null = 1610 o null ≠ 1610?

SQL: lógica de tres valores

Tres posibles valores de verdad

true false unknown

Operadores lógicos

	NOT			OR	AND
false	true	false	false	false	false
true	false	false	true	true	false
		true	false	true	false
		true	true	true	true

SQL: lógica de tres valores

Tres posibles valores de verdad

true false unknown

Operadores lógicos

	NOT
false	true
true	false
unknown	unknown

		OR	AND
false	false	false	false
false	true	true	false
true	false	true	false
true	true	true	true
false	unknown	unknown	false
true	unknown	true	unknown
unknown	false	unknown	false
unknown	true	true	unknown
unknown	unknown	unknown	unknown

SQL: lógica de tres valores

Operaciones con null

$$\text{null} \oplus a = a \oplus \text{null} = \text{op}(\text{null}) := \text{null}$$

para cualquier operación $\oplus \in \{+, *, \backslash, \%, \dots\}$
o función op (con algunas excepciones).
Ejemplo: SUM($\cdot$)

Predicados con null

$$\text{null} \sim a = a \sim \text{null} = P(\text{null}) := \text{unknown}$$

para cualquier comparación $\sim \in \{=, \leq, <, \dots\}$
o predicado P (con algunas excepciones).
Ejemplo: IS NULL

SQL: lógica de tres valores

$$\text{null} \otimes a = a \otimes \text{null} = \text{op}(\text{null}) := \text{null}$$
$$\text{null} \sim a = a \sim \text{null} = \text{P}(\text{null}) := \text{unknown}$$

Ejemplos

- $\text{null} + 3 = 5$ unknown
- $(\text{null IS NULL}) - 1 = 0$ true
- $\text{null} = \text{null}$ unknown
- $2 * \text{null} < 2 \text{ AND } (1 + 1 = 0)$ false

SQL: lógica de tres valores

$$\text{null} \otimes a = a \otimes \text{null} = \text{op}(\text{null}) := \text{null}$$
$$\text{null} \sim a = a \sim \text{null} = \text{P}(\text{null}) := \text{unknown}$$

Semántica de WHERE

Una tupla satisface la **selección** (WHERE) en SQL si, y solo si, la tupla hace la condición **verdadera** (true).

¿es unknown lo mismo que false?

SQL: lógica de tres valores

Satellites

name	planet	discover	dyear
Luna	Tierra	null	null
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

¿son estas consultas equivalentes?

```
SELECT name, planet  
FROM Satellites
```

```
SELECT name, planet  
FROM Satellites  
WHERE dyear = 1610 OR dyear <> 1610
```



name	planet
Luna	Tierra
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter
Titán	Saturno
Triton	Neptuno

null = 1610 OR null <> 1610 = unknown OR unknown
= unknown



SQL: lógica de tres valores

Satellites				Subquery
name	planet	discover	dyear	dyear
Luna	Tierra	null	null	null
Ganímedes	Júpiter	Galileo	1610	1610
Calisto	Júpiter	Galileo	1610	1610
Europa	Júpiter	Galileo	1610	1610
Ío	Júpiter	Galileo	1610	1610
Titán	Saturno	Huygens	1655	
Triton	Neptuno	Lassell	1845	

¿cuál es el resultado de esta consulta?

```
SELECT name, planet
```

```
FROM Satellites
```

```
WHERE dyear NOT IN ( SELECT dyear
```

```
                      FROM Satellites
```

```
                      WHERE planet = 'Tierra' OR
```

```
                        planet = 'Júpiter')
```

} Subquery

¡el resultado es **vacío**!

SQL: lógica de tres valores

Satellites				Subquery
name	planet	discover	dyear	dyear
Luna	Tierra	null	null	null
Ganímedes	Júpiter	Galileo	1610	1610
Calisto	Júpiter	Galileo	1610	1610
Europa	Júpiter	Galileo	1610	1610
Ío	Júpiter	Galileo	1610	1610
Titán	Saturno	Huygens	1655	
Triton	Neptuno	Lassell	1845	

En cámara lenta, para todo $X \neq 1610$

$X \text{ NOT IN } \{\text{null}, 1610\} = \text{NOT } (X \text{ IN } \{\text{null}, 1610\})$
= **NOT** ($X = \text{null}$ **OR** $X = 1610$)
= **NOT** (unknown **OR** false)
= **NOT** (unknown)
= unknown

¿no hay otra **alternativa mejor** para SQL y null?

Outline

Nulos en SQL

Información incompleta

Outer Joins

Información incompleta

Satellites

name	planet	discover	dyear
Luna	Tierra	null	null
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Posibles mundos

Satellites

name	planet	discover	dyear
Luna	Tierra	Eva	4004 AC
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Satellites

name	planet	discover	dyear
Luna	Tierra	Galileo	1609
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

...

¿cómo hacemos esta noción rigurosa?

ADVERTENCIA!

Presentaremos la teoría de **bases de datos incompletas** para bases de datos con **una sola relación**.

(todo se extiende fácilmente a esquemas y varias relaciones)

Tablas de Codd

Sea **Data** el conjunto de **datos**, **Att** un conjunto infinito de **atributos** y **Var** un conjunto infinito de **variables**.

Definición

Una **tabla de Codd** $\mathcal{T}$ es un subconjunto finito de tuplas con los mismos atributos con valores en **Data** y **Var**, y donde **cada variable aparece una sola vez**.

Ejemplo de tabla de Codd

pid	name	cid
1	Ana	x
y	z	102
3	Zoe	v

$$\mathcal{T} = \{ (\text{pid} : 1, \text{name} : \text{Ana}, \text{cid} : x), \\ (\text{pid} : y, \text{name} : z, \text{cid} : 102), \\ (\text{pid} : 3, \text{name} : \text{Zoe}, \text{cid} : v) \}$$

$$\text{att}(\mathcal{T}) = \{\text{pid}, \text{name}, \text{cid}\}$$

Tablas de Codd

Sea **Data** el conjunto de **datos**, **Att** un conjunto infinito de **atributos** y **Var** un conjunto infinito de **variables**.

Definición

Una **tabla de Codd** $\mathcal{T}$ es un subconjunto finito de tuplas con los mismos atributos con valores en **Data** y **Var**, y donde **cada variable aparece una sola vez**.

Definiciones

- Sea $\text{Var}(\mathcal{T}) = \{\bar{\tau}(A) \in \mathbf{Var} \mid A \in \text{att}(\mathcal{T})\}$ todas las variables en $\mathcal{T}$.
- Una **valuación** de $\mathcal{T}$ es una función $\nu : \text{Var}(\mathcal{T}) \cup \mathbf{Data} \rightarrow \mathbf{Data}$ tal que $\nu(a) = a$ para todo $a \in \mathbf{Data}$.
- Para una valuación ν de $\mathcal{T}$ definimos la **relación** $\nu(\mathcal{T})$ tal que:

$$\nu(\mathcal{T}) = \{\nu \circ \bar{\tau} \mid \bar{\tau} \in \mathcal{T}\}$$

Tablas de Codd

Definiciones

- Sea $\text{Var}(\mathcal{T}) = \{\bar{\tau}(A) \in \mathbf{Var} \mid A \in \text{att}(\mathcal{T})\}$ todas las variables en $\mathcal{T}$.
- Una **valuación** de $\mathcal{T}$ es una función $\nu : \text{Var}(\mathcal{T}) \cup \mathbf{Data} \rightarrow \mathbf{Data}$ tal que $\nu(a) = a$ para todo $a \in \mathbf{Data}$.
- Para una valuación ν de $\mathcal{T}$ definimos la **relación** $\nu(\mathcal{T})$ tal que:

$$\nu(\mathcal{T}) = \{\nu \circ \bar{\tau} \mid \bar{\tau} \in \mathcal{T}\}$$

Ejemplo de valuación

$$\nu = (x : 101, y : 2, z : \text{Ben}, v : 102)$$

$\mathcal{T} =$	pid	name	cid
	1	Ana	x
	y	z	102
	3	Zoe	v

$\nu(\mathcal{T}) =$	pid	name	cid
	1	Ana	101
	2	Ben	102
	3	Zoe	102

Tablas de Codd: posibles mundos

Definiciones

- Sea $\text{Var}(\mathcal{T}) = \{\bar{\tau}(A) \in \mathbf{Var} \mid A \in \text{att}(\mathcal{T})\}$ todas las variables en $\mathcal{T}$.
- Una **valuación** de $\mathcal{T}$ es una función $\nu : \text{Var}(\mathcal{T}) \cup \mathbf{Data} \rightarrow \mathbf{Data}$ tal que $\nu(a) = a$ para todo $a \in \mathbf{Data}$.
- Para una valuación ν de $\mathcal{T}$ definimos la **relación** $\nu(\mathcal{T})$ tal que:

$$\nu(\mathcal{T}) = \{\nu \circ \bar{\tau} \mid \bar{\tau} \in \mathcal{T}\}$$

Definición

Para una tabla de Codd $\mathcal{T}$, definimos los **posibles mundos** como:

$$\text{Worlds}(\mathcal{T}) = \{\nu(\mathcal{T}) \mid \nu \text{ es una valuación de } \mathcal{T}\}$$

Una tabla de Codd $\mathcal{T}$ es una **representación finita** para $\text{Worlds}(\mathcal{T})$

Tablas de Codd: posibles mundos

Definición

Para una tabla de Codd $\mathcal{T}$, definimos los **posibles mundos** como:

$$\text{Worlds}(\mathcal{T}) = \{\nu(\mathcal{T}) \mid \nu \text{ es una valuación de } \mathcal{T}\}$$

Ejemplo de posibles mundos

$$\mathcal{T} =$$

pid	name	cid
1	Ana	x
y	z	102
3	Zoe	v

$$\text{Worlds}(\mathcal{T}) = \left\{ \begin{array}{|c|c|c|} \hline \text{pid} & \text{name} & \text{cid} \\ \hline 1 & \text{Ana} & 101 \\ 2 & \text{Ben} & 102 \\ 3 & \text{Zoe} & 102 \\ \hline \end{array} , \begin{array}{|c|c|c|} \hline \text{pid} & \text{name} & \text{cid} \\ \hline 1 & \text{Ana} & 201 \\ 4 & \text{Ian} & 102 \\ 3 & \text{Zoe} & 201 \\ \hline \end{array} , \dots \right\}$$

Bases de datos incompletas

Definición

Para una tabla de Codd $\mathcal{T}$, definimos los **posibles mundos** como:

$$\text{Worlds}(\mathcal{T}) = \{\nu(\mathcal{T}) \mid \nu \text{ es una valuación de } \mathcal{T}\}$$

Definición (general)

Una **base de datos incompleta** $\mathcal{I}$ es un conjunto de relaciones (normales) tal que todas tienen los mismos atributos:

$$\forall \mathcal{R}_1, \mathcal{R}_2 \in \mathcal{I}. \text{ att}(\mathcal{R}_1) = \text{ att}(\mathcal{R}_2)$$

Ejemplos de bases de datos incompletas

- $\text{Worlds}(\mathcal{T})$ con $\mathcal{T}$ tabla de Codd
- $\{\mathcal{R}\}$ con $\mathcal{R}$ una relación (normal)
- $\emptyset$

Consultas sobre bases de datos incompletas

Definición

Para una tabla de Codd $\mathcal{T}$, definimos los **posibles mundos** como:

$$\text{Worlds}(\mathcal{T}) = \{\nu(\mathcal{T}) \mid \nu \text{ es una valuación de } \mathcal{T}\}$$

Definición (general)

Una **base de datos incompleta** $\mathcal{I}$ es un conjunto de relaciones (normales) tal que todas tienen los mismos atributos:

$$\forall \mathcal{R}_1, \mathcal{R}_2 \in \mathcal{I}. \text{ att}(\mathcal{R}_1) = \text{ att}(\mathcal{R}_2)$$

Definición

Para una base de datos incompleta $\mathcal{I}$ y una **consulta** Q se define:

$$Q(\mathcal{I}) = \{Q(\mathcal{R}) \mid \mathcal{R} \in \mathcal{I}\}$$

¡ $Q(\mathcal{I})$ **también** es una base de datos incompleta!

Consultas sobre bases de datos incompletas

Definición

Para una base de datos incompleta $\mathcal{I}$ y una **consulta** Q se define:

$$Q(\mathcal{I}) = \{Q(\mathcal{R}) \mid \mathcal{R} \in \mathcal{I}\}$$

Ejemplo de consultas

$$\mathcal{T} =$$

pid	name	cid
1	Ana	x
y	z	102
3	Zoe	v

$$\text{Worlds}(\mathcal{T}) = \left\{ \begin{array}{|c|c|c|} \hline \text{pid} & \text{name} & \text{cid} \\ \hline 1 & \text{Ana} & 101 \\ 2 & \text{Ben} & 102 \\ 3 & \text{Zoe} & 102 \\ \hline \end{array} , \begin{array}{|c|c|c|} \hline \text{pid} & \text{name} & \text{cid} \\ \hline 1 & \text{Ana} & 201 \\ 4 & \text{Ian} & 102 \\ 3 & \text{Zoe} & 201 \\ \hline \end{array} , \dots \right\}$$

Consultas sobre bases de datos incompletas

Definición

Para una base de datos incompleta $\mathcal{I}$ y una **consulta** Q se define:

$$Q(\mathcal{I}) = \{Q(\mathcal{R}) \mid \mathcal{R} \in \mathcal{I}\}$$

Ejemplo de consultas

$$\text{Worlds}(\mathcal{T}) = \left\{ \begin{array}{|c|c|c|} \hline \text{pid} & \text{name} & \text{cid} \\ \hline 1 & \text{Ana} & 101 \\ 2 & \text{Ben} & 102 \\ 3 & \text{Zoe} & 102 \\ \hline \end{array} , \begin{array}{|c|c|c|} \hline \text{pid} & \text{name} & \text{cid} \\ \hline 1 & \text{Ana} & 201 \\ 4 & \text{Ian} & 102 \\ 3 & \text{Zoe} & 201 \\ \hline \end{array} , \dots \right\}$$

$$\pi_{\text{name}}(\text{Worlds}(\mathcal{T})) = \left\{ \begin{array}{|c|} \hline \text{name} \\ \hline \text{Ana} \\ \text{Ben} \\ \text{Zoe} \\ \hline \end{array} , \begin{array}{|c|} \hline \text{name} \\ \hline \text{Ian} \\ \text{Zoe} \\ \hline \end{array} , \dots \right\} = \text{Worlds} \left(\begin{array}{|c|} \hline \text{name} \\ \hline \text{Ana} \\ \times \\ \text{Zoe} \\ \hline \end{array} \right)$$

Representación de bases de datos incompletas

Pregunta

¿es verdad que para toda tabla de Codd $\mathcal{T}$ y consulta Q ,
existe una tabla de Codd $\mathcal{T}'$ tal que: $Q(\text{Worlds}(\mathcal{T})) = \text{Worlds}(\mathcal{T}')$?

En otras palabras, queremos $\mathcal{T}'$ que **representa** $Q(\mathcal{T})$.

Contraejemplo

$$\mathcal{T} = \begin{array}{cc} \hline \mathbf{A} & \mathbf{B} \\ \hline 0 & 1 \\ x & 2 \\ \hline \end{array} \quad Q = \sigma_{\mathbf{A}=3}(T) \quad Q(\text{Worlds}(\mathcal{T})) = \left\{ \emptyset, \begin{array}{cc} \hline \mathbf{A} & \mathbf{B} \\ \hline 3 & 2 \\ \hline \end{array} \right\}$$

Ejercicio: Demuestre que NO existe tabla de Codd que represente $Q(\mathcal{T})$.

Información incompleta

Área de **investigación** en bases de datos que estudia:

- Representaciones
- Lenguajes de consultas
- Algoritmos eficientes para evaluar consultas

para **bases de datos incompletas**.

Más info: Foundations of databases - Capítulo 19

Outline

Nulos en SQL

Información incompleta

Outer Joins

Datos opcionales

Planets

name	grav	...
Mercurio	2.8	...
Venus	8.9	...
Tierra	9.8	...
Marte	3.7	...
Júpiter	22.9	...
Saturno	9.1	...
Urano	7.8	...
Neptuno	11.0	...

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Encuentre todos los planetas y sus aterrizajes

```
SELECT name, spaceship  
FROM Planets JOIN Landings  
ON (name=planet)
```



name	spaceship
Mercurio	Messenger
Venus	Venera 3
Venus	Pioneer
Marte	Mars 2
Marte	Viking 1
Marte	Beagle 2
Júpiter	Galileo

¿dónde están todos los planetas?

Outer-joins: operador de tuplas

Sea **Data** el conjunto de **datos** con $\text{null} \notin \mathbf{Data}$.

Recordatorio

Si $\bar{\tau}_1$ y $\bar{\tau}_2$ son **compatibles** ($\bar{\tau}_1 \sim \bar{\tau}_2 : \forall A \in \text{att}(\bar{\tau}_1) \cap \text{att}(\bar{\tau}_2). \bar{\tau}_1(A) = \bar{\tau}_2(A)$):

$$[\bar{\tau}_1 \bowtie \bar{\tau}_2](A) := \begin{cases} \bar{\tau}_1(A) & \text{si } A \in \text{att}(\bar{\tau}_1) \\ \bar{\tau}_2(A) & \text{si } A \in \text{att}(\bar{\tau}_2) \end{cases}$$

Definición

Para una tupla $\bar{\tau}$ y un conjunto X de atributos, se define la tupla:

$$\bar{\tau}[X] : \text{att}(\bar{\tau}_1) \cup X \rightarrow \mathbf{Data} \cup \{\text{null}\}$$

tal que para todo $A \in \text{att}(\bar{\tau}_1) \cup X$:

$$\bar{\tau}[X](A) := \begin{cases} \bar{\tau}(A) & \text{si } A \in \text{att}(\bar{\tau}) \\ \text{null} & \text{si } A \notin \text{att}(\bar{\tau}) \end{cases}$$

Outer-joins: operador de tuplas

Sea **Data** el conjunto de **datos** con $\text{null} \notin \mathbf{Data}$.

Definición

Para una tupla $\bar{\tau}$ y un conjunto X de atributos, se define la tupla:

$$\bar{\tau}[X] : \text{att}(\bar{\tau}) \cup X \rightarrow \mathbf{Data} \cup \{\text{null}\}$$

tal que para todo $A \in \text{att}(\bar{\tau}) \cup X$:

$$\bar{\tau}[X](A) := \begin{cases} \bar{\tau}(A) & \text{si } A \in \text{att}(\bar{\tau}) \\ \text{null} & \text{si } A \notin \text{att}(\bar{\tau}) \end{cases}$$

¿cuál es el resultado de $\bar{\tau}[\{\mathbf{pid}, \mathbf{job}\}]$?

$(\mathbf{pid} : 2, \mathbf{name} : \text{Ben}, \mathbf{cid} : 102)[\{\mathbf{pid}, \mathbf{job}\}] =$

$(\mathbf{pid} : 2, \mathbf{name} : \text{Ben}, \mathbf{cid} : 102, \mathbf{job} : \text{null})$

Left outer-join: operador

Para una tupla $\bar{\tau}$ y un conjunto X de atributos, se define la tupla:

$$\bar{\tau}[X](A) := \begin{cases} \bar{\tau}(A) & \text{si } A \in \text{att}(\bar{\tau}) \\ \text{null} & \text{si } A \notin \text{att}(\bar{\tau}) \end{cases}$$

Definición

Para relaciones $\mathcal{R}_1$ y $\mathcal{R}_2$ se define el **left outer-join**:

$$\mathcal{R}_1 \bowtie \mathcal{R}_2 := \{ \bar{\tau}_1 \bowtie \bar{\tau}_2 \mid \bar{\tau}_1 \in \mathcal{R}_1 \wedge \bar{\tau}_2 \in \mathcal{R}_2 \wedge \bar{\tau}_1 \sim \bar{\tau}_2 \} \cup \underbrace{\{ \bar{\tau}_1[\text{att}(\mathcal{R}_2)] \mid \bar{\tau}_1 \in \mathcal{R}_1 \wedge \forall \bar{\tau}_2 \in \mathcal{R}_2. \bar{\tau}_1 \not\sim \bar{\tau}_2 \}}_{\text{tuplas en } \mathcal{R}_1 \text{ que } \mathbf{NO} \text{ son compatibles} \\ \text{con } \mathbf{ninguna} \text{ tupla en } \mathcal{R}_2}$$

Left outer-join: operador

Para una tupla $\bar{\tau}$ y un conjunto X de atributos, se define la tupla:

$$\bar{\tau}[X](A) := \begin{cases} \bar{\tau}(A) & \text{si } A \in \text{att}(\bar{\tau}) \\ \text{null} & \text{si } A \notin \text{att}(\bar{\tau}) \end{cases}$$

Definición

Para relaciones $\mathcal{R}_1$ y $\mathcal{R}_2$ se define el **left outer-join**:

$$\begin{aligned} \mathcal{R}_1 \bowtie \mathcal{R}_2 := & \{ \bar{\tau}_1 \bowtie \bar{\tau}_2 \mid \bar{\tau}_1 \in \mathcal{R}_1 \wedge \bar{\tau}_2 \in \mathcal{R}_2 \wedge \bar{\tau}_1 \sim \bar{\tau}_2 \} \cup \\ & \{ \bar{\tau}_1[\text{att}(\mathcal{R}_2)] \mid \bar{\tau}_1 \in \mathcal{R}_1 \wedge \forall \bar{\tau}_2 \in \mathcal{R}_2. \bar{\tau}_1 \not\sim \bar{\tau}_2 \} \end{aligned}$$

Ejemplo de $\bowtie$

pid	name	cid		pid	job		pid	name	cid	job
1	Ana	101	$\bowtie$	1	Guía	$=$	1	Ana	101	Guía
2	Ben	102		2	Chef		2	Ben	102	Chef
3	Zoe	102		2	Guía		2	Ben	102	Guía
				4	Poeta		3	Zoe	102	null

Right outer-join: operador

Para una tupla $\bar{\tau}$ y un conjunto X de atributos, se define la tupla:

$$\bar{\tau}[X](A) := \begin{cases} \bar{\tau}(A) & \text{si } A \in \text{att}(\bar{\tau}) \\ \text{null} & \text{si } A \notin \text{att}(\bar{\tau}) \end{cases}$$

Definición

Para relaciones $\mathcal{R}_1$ y $\mathcal{R}_2$ se define el **right outer-join**:

$$\begin{aligned} \mathcal{R}_1 \bowtie \mathcal{R}_2 := & \{ \bar{\tau}_1 \bowtie \bar{\tau}_2 \mid \bar{\tau}_1 \in \mathcal{R}_1 \wedge \bar{\tau}_2 \in \mathcal{R}_2 \wedge \bar{\tau}_1 \sim \bar{\tau}_2 \} \cup \\ & \{ \bar{\tau}_2[\text{att}(\mathcal{R}_1)] \mid \bar{\tau}_2 \in \mathcal{R}_2 \wedge \forall \bar{\tau}_1 \in \mathcal{R}_1. \bar{\tau}_1 \not\sim \bar{\tau}_2 \} \end{aligned}$$

Ejemplo de $\bowtie$

pid	name	cid		pid	job		pid	name	cid	job
1	Ana	101	$\bowtie$	1	Guía	=	1	Ana	101	Guía
2	Ben	102		2	Chef		2	Ben	102	Chef
3	Zoe	102		2	Guía		2	Ben	102	Guía
				4	Poeta		4	null	null	Poeta

Right outer-join: operador

Para una tupla $\bar{\tau}$ y un conjunto X de atributos, se define la tupla:

$$\bar{\tau}[X](A) := \begin{cases} \bar{\tau}(A) & \text{si } A \in \text{att}(\bar{\tau}) \\ \text{null} & \text{si } A \notin \text{att}(\bar{\tau}) \end{cases}$$

Definición

Para relaciones $\mathcal{R}_1$ y $\mathcal{R}_2$ se define el **right outer-join**:

$$\begin{aligned} \mathcal{R}_1 \ltimes \mathcal{R}_2 := & \{ \bar{\tau}_1 \bowtie \bar{\tau}_2 \mid \bar{\tau}_1 \in \mathcal{R}_1 \wedge \bar{\tau}_2 \in \mathcal{R}_2 \wedge \bar{\tau}_1 \sim \bar{\tau}_2 \} \cup \\ & \{ \bar{\tau}_2[\text{att}(\mathcal{R}_1)] \mid \bar{\tau}_2 \in \mathcal{R}_2 \wedge \forall \bar{\tau}_1 \in \mathcal{R}_1. \bar{\tau}_1 \not\sim \bar{\tau}_2 \} \end{aligned}$$

$$\mathcal{R}_1 \ltimes \mathcal{R}_2 \equiv \mathcal{R}_2 \bowtie \mathcal{R}_1$$

Full outer-join: operador

Definición

Para relaciones $\mathcal{R}_1$ y $\mathcal{R}_2$ se define el **full outer-join**:

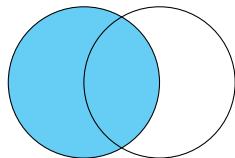
$$\mathcal{R}_1 \bowtie \mathcal{R}_2 := \mathcal{R}_1 \bowtie \mathcal{R}_2 \cup \mathcal{R}_1 \ltimes \mathcal{R}_2$$

Ejemplo de $\bowtie$

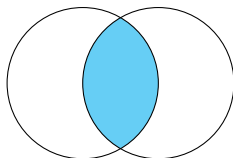
pid	name	cid		pid	job		pid	name	cid	job
1	Ana	101	$\bowtie$	1	Guía	=	1	Ana	101	Guía
2	Ben	102		2	Chef		2	Ben	102	Chef
3	Zoe	102		2	Guía		2	Ben	102	Guía
				4	Poeta		3	Zoe	102	null
							4	null	null	Poeta

Outer-joins nos permite extender tuplas con **datos opcionales**

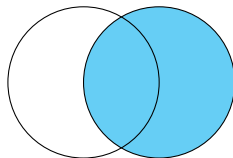
Resumen de outer-joins



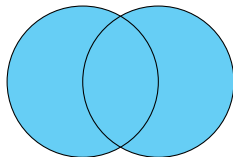
Left outer-join



Join



Right outer-join



Full outer-join

Outer-joins en SQL

Planets

name	grav	...
Mercurio	2.8	...
Venus	8.9	...
Tierra	9.8	...
Marte	3.7	...
Júpiter	22.9	...
Saturno	9.1	...
Urano	7.8	...
Neptuno	11.0	...

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Encuentre todos los planetas y sus aterrizajes

```
SELECT name, spaceship  
FROM Planets LEFT OUTER JOIN  
Landings ON (name=planet)
```



name	spaceship
Tierra	null
Saturno	null
Urano	null
Neptuno	null
Mercurio	Messenger
Venus	Venera 3
Venus	Pioneer
Marte	Mars 2
Marte	Viking 1
Marte	Beagle 2
Júpiter	Galileo

Outer-joins en SQL

Planets			Landings			
name	grav	...	spaceship	planet	ctry	year
Mercurio	2.8	...	Messenger	Mercurio	EEUU	2015
Venus	8.9	...	Venera 3	Venus	URSS	1966
Tierra	9.8	...	Pioneer	Venus	EEUU	1978
Marte	3.7	...	Mars 2	Marte	URSS	1971
Júpiter	22.9	...	Viking 1	Marte	EEUU	1976
Saturno	9.1	...	Beagle 2	Marte	ESA	2003
Urano	7.8	...	Galileo	Júpiter	EEUU	2003
Neptuno	11.0	...				

Encuentre todos los planetas y sus aterrizajes

```
SELECT name, spaceship  
FROM Planets LEFT OUTER JOIN  
Landings ON (name=planet)
```



name	spaceship
Tierra	null
Saturno	null
Urano	null
Neptuno	null
Mercurio	Messenger
Venus	Venera 3
...	...

SQL también soporta RIGHT OUTER JOIN y FULL OUTER JOIN.